

Connected Vehicle Intelligence Hackathon

Solution Document: FleetPulse

Submission Format: PDF (export of this document)

To be Submitted by: VOOTKUR VENNELA REDDY(RA2311003011203)

Team Members & Roles: VOOTKUR VENNELA REDDY-Solo developer (design, development, testing, documentation)

Problem Space Chosen: Predictive maintenance for mixed-maker fleets, with multi-OEM data normalisation as its enabler (own framing of two listed problem spaces)

Repository URL: <https://github.com/vennela9927/fleetPulseproject>

Demo Video URL (≤ 5 min):

<https://drive.google.com/file/d/1xHAFBI4Agx50tX9YDKDF9llaxLwQ8bW9/view?usp=sharing>

Date of Submission: 02/10/2026

Table of Contents

- [1. Executive Summary](#)
- [2. Problem Statement & Validation](#)
 - [2.1 Problem Statement](#)
 - [2.2 Evidence & Validation](#)
 - [2.3 Impact & Success Metrics](#)
- [3. Solution Description](#)
 - [3.1 Solution Overview & User Journey](#)
 - [3.2 Key Value Proposition](#)
 - [3.3 Innovative Ideas](#)
- [4. Feature List](#)
- [5. Solution Architecture \(High-Level Design\)](#)
 - [5.1 Architecture Overview](#)
 - [5.2 Technology Stack & Justification](#)
 - [5.3 Data Architecture](#)
 - [5.4 Deployment View](#)
- [6. Low-Level Design](#)
 - [6.1 Layering & Separation of Concerns](#)
 - [6.2 Design Principles Applied](#)
 - [6.3 Design Patterns Used](#)
 - [6.4 Interfaces, Contracts & Runtime Flows](#)
 - [6.5 Algorithms & Data Structures](#)
- [7. Non-Functional Requirements & Performance Benchmarks](#)
- [8. Security & Compliance](#)
- [9. Test Strategy](#)
- [10. Observability](#)
- [11. AI / ML Component \(if used\)](#)
- [12. Architecture Decisions, Risks & Future Enhancements](#)
- [13. Demo Video \(5 Minutes Maximum\)](#)
- [14. Repository Checklist](#)
- [15. Conclusion](#)
- [16. Declarations](#)
- [17. Appendix \(if any\)](#)

1. Executive Summary

The problem. A fleet that buys vehicles from several makers receives several telemetry formats and still loses vehicles to breakdowns whose warning signs were in that data. Fleet managers pay for towing, downtime and emergency repairs, and have no single, trustworthy view across makers to plan maintenance from.

What FleetPulse does. It turns telemetry from any maker into one standard event, raises live alerts within about half a second, predicts **which vehicles will break down in the next 7 days**, and turns that into a workshop plan within depot capacity that a manager books directly or approves when an AI copilot proposes it.

Key results (all on simulated data from our own 100,000-vehicle simulator, four makers; details and conditions in sections 7, 9 and 11):

- **Prediction:** on vehicles and later days never seen in training, the calibrated model raised alerts of which **77.8% were real breakdowns** and caught **78.0% of breakdowns**, for a net saving of **+\$712K per week per 100K vehicles**; a mechanic's rule at its own volume lost \$466K. Accuracy held for a maker left out of training (PR-AUC drop ≤ 0.011).
- **Nothing lost in the failure we tested:** Kafka broker hard-killed mid-stream, then reconciled: **2,874,026 events sent = stored = counted in the rollup** (no loss, no double count).
- **Latency and throughput on one laptop:** critical alerts p50 0.44 s / p99 0.63 s; 3,300 events/s sustained (100K vehicles every 30 s), 8,710 events/s stored during a 10K/s burst; Kafka alone 40,900 records/s.
- **Query optimisation:** the three slowest API queries went from 140 ms–1.9 s to 0.6–9 ms (measured EXPLAIN ANALYZE, section 5.3).

What is new: a new maker is onboarded with a mapping file, previewed on its own parked traffic and approved live with no restart and no data lost; alerts fire on a cost rule the fleet sets, not a tuned threshold; and the platform watches its own trustworthiness (sensor plausibility, per-maker feed drift that recognised “mph sent as km/h”, and a model graded by workshop results).

2. Problem Statement & Validation

2.1 Problem Statement

“A **fleet manager** running vehicles from several makers **needs a way to** know which vehicles will break down in the next 7 days and get them inspected first, **because** the warning signs are spread across maker-specific data formats nobody reads together, **which today costs** a towed vehicle, lost working days and a repair premium for every breakdown that could have been a planned visit.” In our simulated fleet, 3,669 breakdowns occurred in 21 days across 98,000 vehicles, and catching one in advance saved **\$902 on average** over its planned repair.

Primary user: the fleet manager (daily inspection plan, bookings, approvals).

Secondary stakeholders: depot and workshop leads (bay capacity, parts, inspection results), the platform team (onboarding new makers, data quality), drivers (fewer roadside breakdowns), finance (avoided cost).

2.2 Evidence & Validation

Facts vs assumptions. Facts below come from the problem statement or from measurements on our system. Business assumptions about real fleets are labelled as such: we did not have access to real fleet data or interviews, so every result is validated **by simulation** on data with realistic structure.

Evidence / Assumption	Source or Method	What It Shows	Confidence
Fact: connected fleets mix makers; Motorq maps data from 25+ brands into one schema	Problem statement §2–3	Normalisation is a prerequisite for any cross-maker analytics	High
Fact: up to ~25 GB/hour per car; most data must be filtered before the cloud	Problem statement §3	Only useful, normalised signal should reach storage and models	High
Assumption: an unplanned breakdown costs more than a planned repair of the same part	Simulator: breakdown = 1.6 × repair cost (towing, downtime)	Value of predicting early: \$902 average per caught breakdown	Medium
Assumption: faults show warning signs days before failure (codes, drifting voltage, heat, battery health)	Simulator fault model: 3–10 days of degradation; 15% of faults silent	Prediction is possible but not perfect; median warning 2.7 days	Medium
Fact (measured): rule-based alerts on fault codes and thresholds flag many healthy vehicles	Back-test of a mechanic's rule on held-out data	8% of flagged vehicles broke down; ~\$466K/week per 100K	High (on simulated data)
Fact (measured): firmware can change units without breaking validation	Live test: Aurora speed sent in mph labelled km/h	Values pass every range check; only drift monitoring caught it	High

Existing alternatives and where they fall short:

- **Aftermarket OBD-II dongles:** hardware to buy, install and maintain per vehicle; one more data source rather than one view across makers.
- **OEM portals:** one portal per maker, each with its own format and alerts; no fleet-wide ranking or plan.
- **Motorq Fuse** (public description: an AI layer that flags issues such as idling and estimates their dollar impact): we did not have access to it, so we do not compare results. FleetPulse focuses on calibrated per-vehicle breakdown probability turned into a bay-constrained workshop plan.
- **Rule-based telematics alerts:** our measured baseline. At the same number of vehicles flagged per day, the model caught 80% of breakdowns against the rule's 54%.

2.3 Impact & Success Metrics

Metric	Baseline Today	Target	Achieved / How Measured
Breakdowns caught in advance (7-day horizon)	53.9% (mechanic's rule)	≥ 75%	78.0% (cost rule); back-test on 20% unseen vehicles, later days
Alerts that were real breakdowns	8.0% (rule)	≥ 50%	77.8%; same back-test

Metric	Baseline Today	Target	Achieved / How Measured
Net saving per week per 100K vehicles	-\$466K (rule)	> 0	+\$712K at \$150 per inspection; same back-test
Events lost or double-counted when a broker dies	Unknown	0	0 of 2,874,026 (scripted broker kill)
Time to onboard a new maker's format	Code change + redeploy	Minutes, no restart	Mapping live on all instances within ~1 s of approval; parked data replayed (tested live with Draco)
Critical alert latency (reading → stored alert)	–	< 5 s	p50 0.44 s, p99 0.63 s (58 alerts)

Scale of impact. The saving is per vehicle: at the same failure rate and \$150 inspection cost, +\$712K per week per 100K vehicles is about **+\$71K per week per 10K**. This is linear by assumption; a real fleet's failure rate and repair costs would change it, which is why the inspection cost is a setting on the page, not a constant.

Wider impact. Safety: fewer roadside breakdowns and live harsh-driving and overheating alerts. Cost: inspections only where the expected saving exceeds their cost. Compliance: tenant isolation enforced by the databases, a hash-chained audit log of every booking and AI action, driver data kept apart from telemetry.

3. Solution Description

3.1 Solution Overview & User Journey

What it does. FleetPulse collects telemetry from four makers' clouds, converts it to one standard event, and uses it twice: live (alerts, map, vehicle state within about a second) and in batch (21 days of history train a breakdown model that scores every vehicle daily). The manager sees whom to inspect, where and when, and books it.

User journey (vehicle event → detection → alert → action → outcome):

- A Borealis truck sends speed in mph and coolant in °F in its own JSON; the gateway accepts it only after Kafka has stored it, and the normaliser converts it to the standard event.
- Live rules catch an overheating engine within about half a second and show it in the alert feed; the nightly model notices its fault codes, voltage and temperature trends rising.
- **Fleet health** shows 350 vehicles to inspect tomorrow; the vehicle page explains its 96% risk, the likely part and the money at stake.
- The **service plan** gives it a bay at its home depot tomorrow; the manager books, or the copilot proposes and the manager approves, and the audit trail records both.
- The mechanic records what the inspection found; the model is graded against the probabilities it promised.

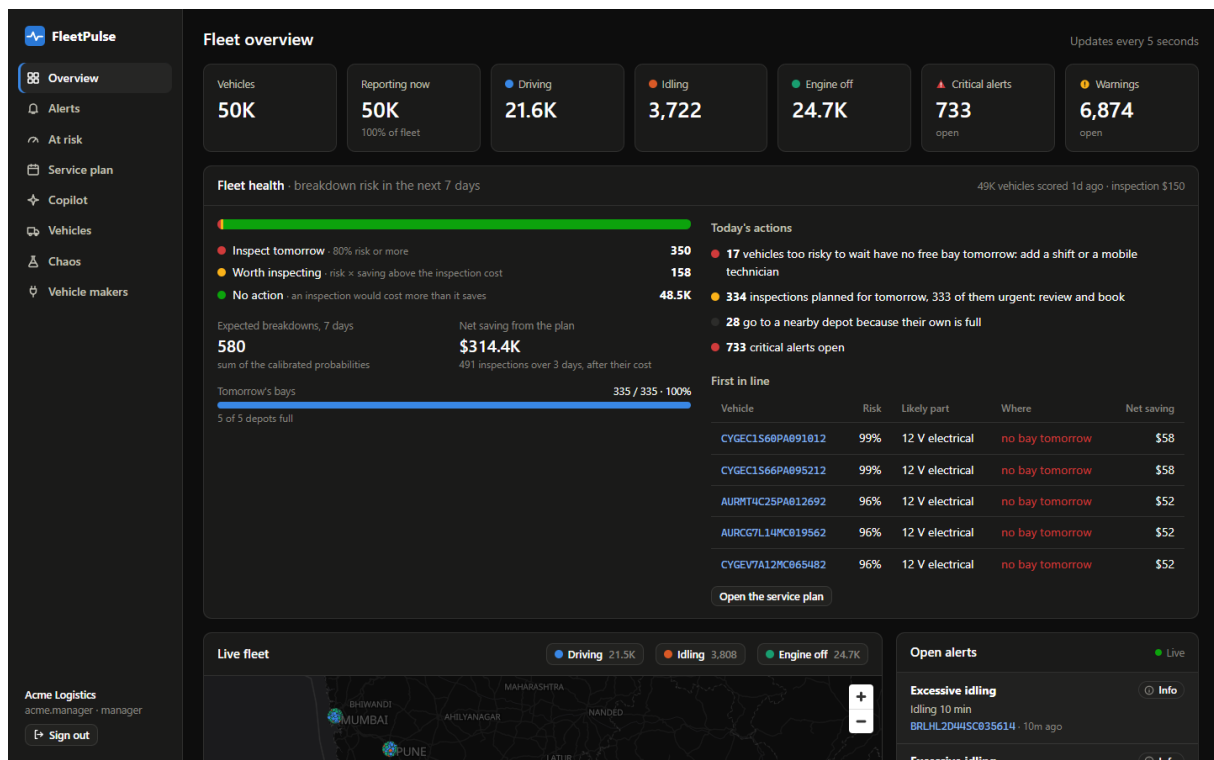


Figure 1. Overview with Fleet health: whom to inspect tomorrow, bays, today's actions, first in line (Acme Logistics, 50K vehicles).

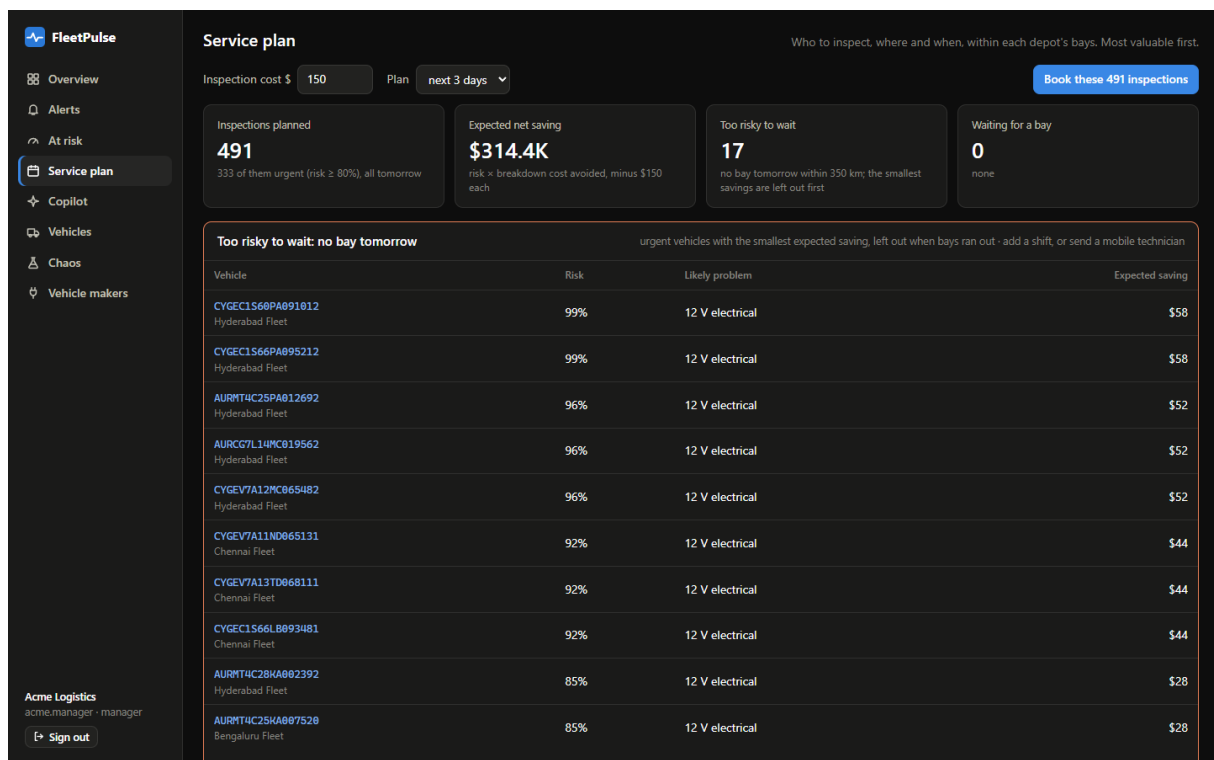


Figure 2. Service plan: 491 inspections within depot bays, \$314.4K expected net saving, and urgent vehicles with no bay reported as too risky to wait.

3.2 Key Value Proposition

	FleetPulse
Customer job	Keep vehicles on the road at the lowest maintenance cost across a mixed-maker fleet.

	FleetPulse
Pain relieved	No more reading each maker's portal; no flood of rule alerts (8% real) to triage; fewer tow trucks and emergency repairs.
Gain created	A ranked, explained 7-day risk per vehicle, a bookable plan within bay capacity, a parts forecast per depot, and a copilot that can prepare bookings but never execute them alone.
Differentiation	Works the same for a maker the model never saw (features use the standard event); new makers onboard in minutes from a mapping file; the decision rule is the fleet's own inspection cost; every number shown is reconciled or back-tested.

3.3 Innovative Ideas

Idea	What is new (compared with what we studied)	Evidence it works
1. Onboard a maker live from a mapping file	Common practice is code per maker and a redeploy. Here a declarative mapping is previewed on that maker's own parked traffic with the production engine, approved, and picked up by every instance from a compacted Kafka topic; data sent before onboarding is replayed, not lost.	Draco (2,000 trucks) onboarded live in testing with no restart; normaliser topology test parks then maps the same payload; contract test maps 400 events × 8 models back to their source values.
2. Alert on the fleet's own cost, from calibrated probabilities	Instead of a tuned threshold: flag when probability × average saving > inspection cost, then allocate bays greedily by value. A precision target was tried first and rejected (it chose 0.5%).	+\$712K/week per 100K at \$150; within 1% of the best threshold chosen in hindsight at \$300; predictions of 80–100% came true 93.6% of the time.
3. A platform that checks its own trustworthiness	Sensor-plausibility rule (a coolant sensor cannot jump 25 °C between readings); per-maker drift relative to the other makers with unit hints; workshop results grading the model's promises.	3 broken sensors flagged after three impossible readings (about 1 minute at a 30 s reporting interval), with no false overheat; Aurora mph-as-km/h bug flagged in 2 min 17 s and named at ~7 min, nothing else flagged; workshop loop caught the 99% cap as overconfident (86% found vs 96% promised).

4. Feature List

Priority uses MoSCoW; status is Done / Partial / Planned.

ID	Feature	User Story	Priority	Status	Code Path
F-01	100K-vehicle simulator: 4 maker formats, faults with warning signs, duplicates, out-of-order, invalid data	As a reviewer I want realistic messy data at scale so that the platform is tested like production	Must	Done	services/simulator/
F-02	Ingest gateway: API key, back-pressure (429/503 + Retry-After), circuit breaker, acks=all	As an OEM partner I want a clear 'retry later' so that I never lose data in an outage	Must	Done	services/ingest-gateway/
F-03	Declarative mapping to one standard event; dead-letter topic with reasons	As the platform team I want every maker in one schema so that analytics are written once	Must	Done	services/common/.../mapping/, services/normalizer/
F-04	Exactly-once normalisation with (VIN, seq) de-duplication	As a manager I want each event counted once so that alerts and reports are right	Must	Done	services/normalizer/ (SeqWindow)
F-05	Live alert rules: overheat, critical fault code, low 12 V, harsh driving, EV low charge, idling, sensor fault	As a fleet manager I want critical alerts within seconds so that I can act now	Must	Done	services/stream-processor/.../alert/
F-06	Live map and vehicle state (Redis read model)	As a manager I want to see where vehicles are now	Must	Done	stream-processor/.../live/, web/src/components/LiveMap.tsx

ID	Feature	User Story	Priority	Status	Code Path
F-07	Exactly-once ClickHouse sink with per-minute and daily rollups	As an analyst I want history that is never double-counted	Must	Done	services/stream-processor/.../sink/
F-08	Tenant isolation inside Postgres (RLS), ClickHouse (row policy) and Redis (keys)	As a customer I want certainty no other company sees my fleet	Must	Done	infra/postgres/init/02_security.sh, services/api/fleetpulse_api/db.py
F-09	Secure API: OIDC (Keycloak), roles, keyset pagination, Redis token-bucket rate limits, RFC 9457 errors	As an integrator I want a stable, documented, protected API	Must	Done	services/api/
F-10	Calibrated 7-day breakdown model with per-vehicle reasons and likely part	As a manager I want to know who will break down and why	Must	Done	services/ml/
F-11	Fleet health overview and today's actions	As a manager I want my day's decisions on one screen	Should	Done	web/src/components/FleetHealth.tsx
F-12	Service plan within depot bays; all-or-nothing booking	As a depot lead I want only bookings that fit my bays	Should	Done	services/api/fleetpulse_api/maintenance.py
F-13	Parts forecast per depot with 90% range	As a depot lead I want to stock the right parts	Could	Done	maintenance.py (parts_forecast)
F-14	Copilot (Gemini, rule fallback) with propose-only tools, approval and audit trail	As a manager I want help preparing bookings without losing control	Should	Done	services/api/fleetpulse_api/copilot/
F-15	Hash-chained, append-only audit log with verification	As an auditor I want to detect any edit	Must	Done	02_security.sh, fleetpulse_api/audit.py
F-16	Live maker onboarding: propose, preview on parked data, approve, replay	As the platform team I want a new maker live in minutes	Should	Done	routers/onboarding.py, normalizer/DlqReplayer.java, web/src/pages/Makers.tsx
F-17	Feed drift per maker (relative PSI, unit hints)	As the platform team I want silent feed changes caught	Could	Done	services/ml/fleetpulse_ml/drift.py
F-18	Workshop results grade the model	As a data owner I want to know when to recalibrate	Could	Done	routers/maintenance.py (inspections, model-health)
F-19	Chaos page: sent = stored reconciliation, fault injection, traffic spikes	As a reviewer I want to see nothing is lost	Should	Done	web/src/pages/Chaos.tsx, routers/ops.py
F-20	Similar past failures (pgvector)	As a mechanic I want to know what fixed similar cases	Could	Done	copilot/tools.py (similar_past_failures)
F-21	One-command compose; Helm (AWS, GCP values); Terraform (AWS)	As an operator I want the same deployment anywhere	Must	Done (Helm/Terraform validated, not applied)	docker-compose.yml, deploy/
F-22	Right-to-erasure endpoint	As a driver I want my personal data erased on request	Should	Partial (schema only)	infra/postgres/init/01_schema.sql (erasure_request)
F-23	Location masking by role	As a privacy officer I want analysts to see coarse locations	Should	Planned	–
F-24	BDD acceptance suite; Semgrep and ZAP scans	As a reviewer I want behaviour-level evidence	Should	Planned	tests/bdd/ (empty)

5. Solution Architecture (High-Level Design)

5.1 Architecture Overview

System context (C4 level 1)

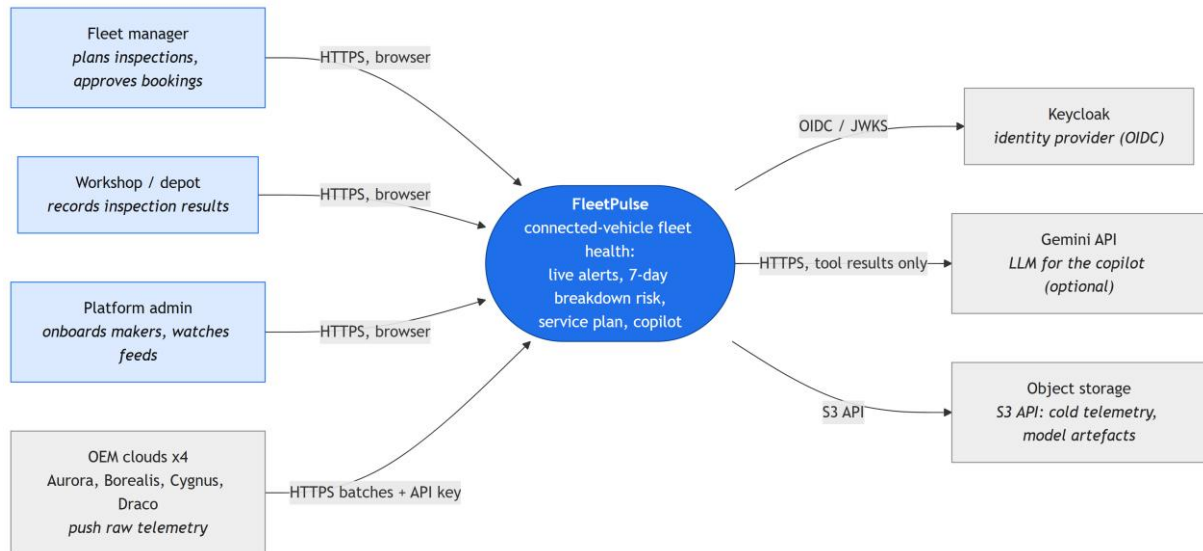


Figure 3. System context: users, OEM clouds and the external systems FleetPulse depends on.

Containers (C4 level 2), with the protocol on each arrow

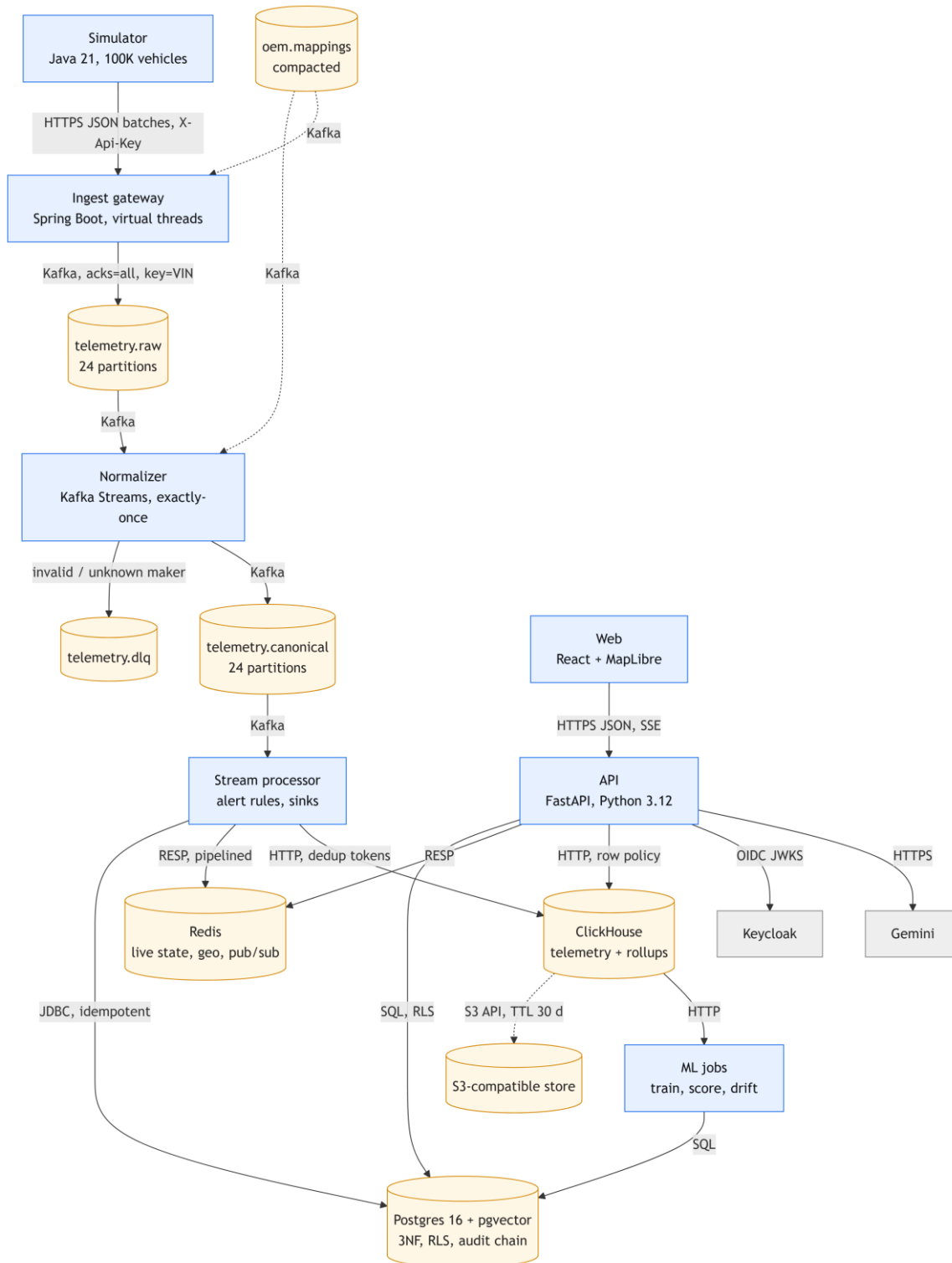


Figure 4. Containers. Kafka is the durable log: every store downstream of it can be rebuilt by replaying it.

Path of one telemetry event, with latency per hop

#	Hop	Latency	Measured or design
1	OEM cloud → gateway → Kafka telemetry.raw (202 only after acks=all)	milliseconds when healthy	Design; gateway p50/p95/p99 exposed as gateway_batch_latency

#	Hop	Latency	Measured or design
2	Normaliser: map + validate + de-duplicate → telemetry.canonical (transaction every 200 ms)	≤ 0.2 s + processing	Design (commit interval)
3	Stream processor: alert rules → Postgres alert + Redis pub/sub	p50 0.44 s, p99 0.63 s from the reading's timestamp to the stored alert	Measured, 58 alerts, steady state
4	Stream processor → Redis live state → dashboard	≤ 5 s (dashboard refresh interval)	Design
5	Stream processor → ClickHouse (5 s batches, dedup tokens) → rollups	≤ 5 s flush	Design (ADR 0001)
6	ClickHouse → nightly feature build → model scores in Postgres	daily batch	Design

5.2 Technology Stack & Justification

Layer	Choice	Why This, and What We Rejected
Ingestion / Messaging	HTTPS batch gateway (Spring Boot 3.3, Java 21 virtual threads); Apache Kafka 3.8 (KRaft), 24 partitions keyed by VIN	OEM clouds push batches over HTTPS (device-free, as in Motorq's model), so MQTT was not needed. Kafka over RabbitMQ for replay (DLQ replay, rebuilding Redis), per-vehicle ordering and transactions for exactly-once. Redpanda rejected: Kafka Streams exactly-once relies on Kafka's transaction semantics.
Stream / Batch Processing	Kafka Streams (exactly-once v2) for normalising and live rules; ClickHouse materialised views and Python jobs (pandas, LightGBM) for history	Kafka Streams is a library inside each service: no extra cluster. Flink and Spark rejected for the time available; ClickHouse SQL aggregates 146M rows without them.
Relational / NoSQL / Cache / Search / Vector	PostgreSQL 16 (3NF, RLS); ClickHouse 24.8 (columnar); Redis 7.4; pgvector; SeaweedFS (S3 API)	One store per access pattern (ADR 0005). A single Postgres rejected (write amplification at 10K events/s); Cassandra rejected (no ad-hoc analytics); a separate vector database rejected (thousands of vectors); MinIO rejected because its images are no longer published.
Backend / Frontend	FastAPI (Python 3.12, async psychopg 3, PyJWT); React 19, Vite, TanStack Query, MapLibre GL, Recharts	Python for the API shares libraries and data code with the ML jobs; Java for the high-throughput pipeline. Server-side rendering was not needed for a signed-in dashboard.
ML / AI	LightGBM + isotonic calibration (scikit-learn); Gemini via function calling with a rule-based fallback; pgvector fault signatures	Gradient-boosted trees handle missing values (EVs have no coolant) and give exact per-feature contributions for the 'why'. Survival and sequence models rejected (ADR 0003). Gemini chosen for its free tier; the copilot works without it.
Infrastructure / CI-CD / Observability	Docker Compose; Helm chart with AWS and GCP values; Terraform for AWS (EKS, RDS, MSK, ElastiCache, S3, ECR); GitHub Actions; Prometheus metrics endpoints	Same images everywhere, configured only by environment. Grafana dashboards and tracing were not built in the time available (section 10).

5.3 Data Architecture

Entity-relationship diagram (relational core, 3NF)

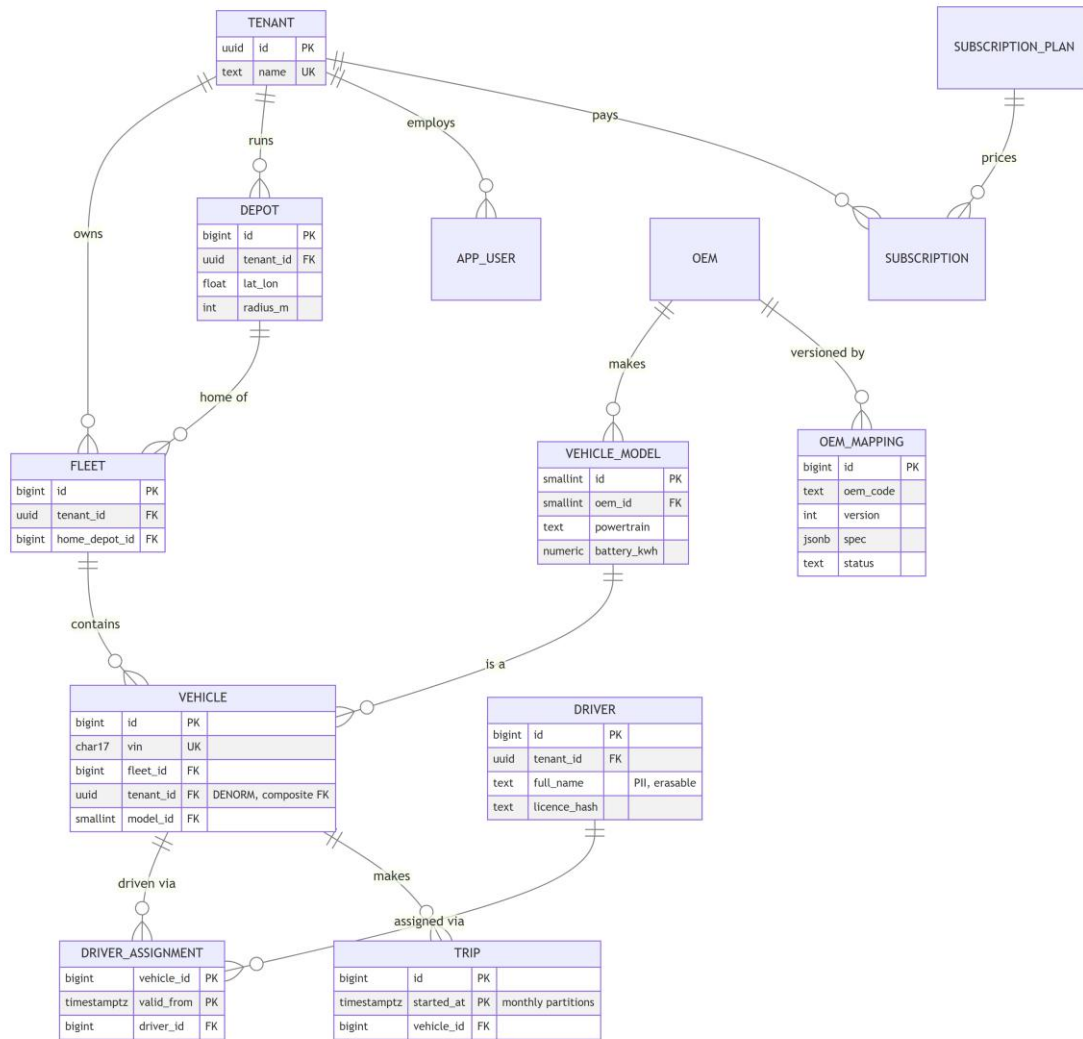


Figure 5a. Tenancy, fleet and vehicle master data (key columns shown).

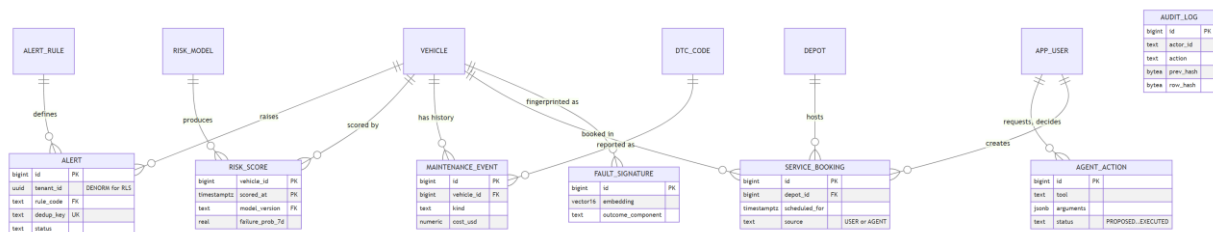


Figure 5b. Alerts, predictions, maintenance, bookings, copilot actions and the audit log. 28 tables in total; full schema in `infra/postgres/init/01_schema.sql`.

Table	Key	Depends on (FK)	Purpose
alert	id; dedup_key unique	vehicle, alert_rule	Live-rule alerts; deterministic key makes the sink idempotent
risk_score	(vehicle_id, scored_at)	vehicle, risk_model	Daily 7-day breakdown probability, part, reasons
maintenance_event	id	vehicle, dtc_code	Breakdowns and repairs: the model's labels
service_booking	id	vehicle, depot, app_user	Inspections, by a user or approved from the copilot
fault_signature	id	vehicle	16-number fingerprints (pgvector) with the outcome

Table	Key	Depends on (FK)	Purpose
agent_action	id	app_user	Copilot proposals and the human decision
audit_log	id (hash-chained)	–	Append-only record of every action, each row hashing the previous

Deliberate denormalisation. `tenant_id` is copied onto `vehicle`, `alert`, `risk_score` and `maintenance_event` so row-level security needs no join. A composite foreign key (`vehicle(fleet_id, tenant_id) → fleet(id, tenant_id)`) makes an inconsistent copy impossible. Model attributes live on `vehicle_model`, not `vehicle`, and drivers relate to vehicles through `driver_assignment` over time.

Polyglot map and CAP choice

Data	Store	CAP / PACELC choice
Tenants, vehicles, bookings, approvals, audit log, risk scores	PostgreSQL 16	CP: a booking and its audit row commit together or not at all; single primary (sync standby in prod)
Raw telemetry, per-minute and daily rollups	ClickHouse	AP-leaning, PACELC 'else latency': may trail Kafka by up to 5 s; exact via replay and dedup tokens
Live state, map, alert fan-out	Redis	AP: seconds-stale reads are acceptable; rebuildable from Kafka
Event log	Kafka	CP for writes (acks=all, min ISR 2 with 3 replicas in prod): refuses a write rather than lose it, which is the gateway's 503 path
Fault signatures (vector)	pgvector in Postgres	CP, joined to vehicles and repairs in one transaction
Cold telemetry, model artefacts	S3-compatible object storage	Durable; written by ClickHouse tiering

Capacity estimate

Quantity	Value	Basis
Design rate	10K events/s (100K vehicles × 1 per 10 s); 30K/s in a 3× burst	Problem statement scale
Event size	~330 B raw JSON; 22 B per row in ClickHouse (6.1× compression)	Measured: 2,000-message sample; 145.9M rows in 2.98 GiB
Daily / yearly volume	864M events/day → ~19 GB/day, ~6.9 TB/year in ClickHouse	Design rate × measured bytes per row
Partition keys	Kafka: VIN (24 partitions, even load: every vehicle reports at the same rate). ClickHouse: one partition per day, sorted by (vin, ts, seq). Postgres trips: monthly range partitions	Schema
Hot / warm / cold	Redis: seconds. Kafka raw 7 days, canonical 3 days. ClickHouse local disk 30 days (~570 GB at design rate), S3 days 30–90 (~1.1 TB), deleted at 90. Daily rollups kept	TTLs in <code>O1_telemetry.sql</code> and <code>create-topics.sh</code>
Storage cost (estimate)	~\$45/month hot disk (gp3) and ~\$25/month cold S3 per ClickHouse copy	Approximate public list prices, us-east-1; excludes compute

Query optimisation (measured, EXPLAIN ANALYZE)

Measured on 2026-10-01 as the API's own role with row-level security, tenant Acme (50,000 vehicles), median of 5 runs, while the simulator was streaming. Full plans: `docs/evidence/query-plans.md`; script: `docs/evidence/bench_queries.py`.

Query	Before (ms)	After (ms)	Change Made
At-risk ranking (top 50 of the latest scoring run)	1,881	2.3	Index risk_score_ranked (tenant, model, run, probability, vehicle) + scalar 'latest run' lookup: index scan stopping at 51 rows instead of two scans of every score and a sort
Vehicle list, page 900 of 50	140	0.6	Keyset pagination (opaque cursor on id) instead of OFFSET 44,950
Latest alerts (top 50)	228	9.1	Index alert_tenant_opened (tenant, opened_at DESC, id DESC): no sort of 42K alerts

5.4 Deployment View

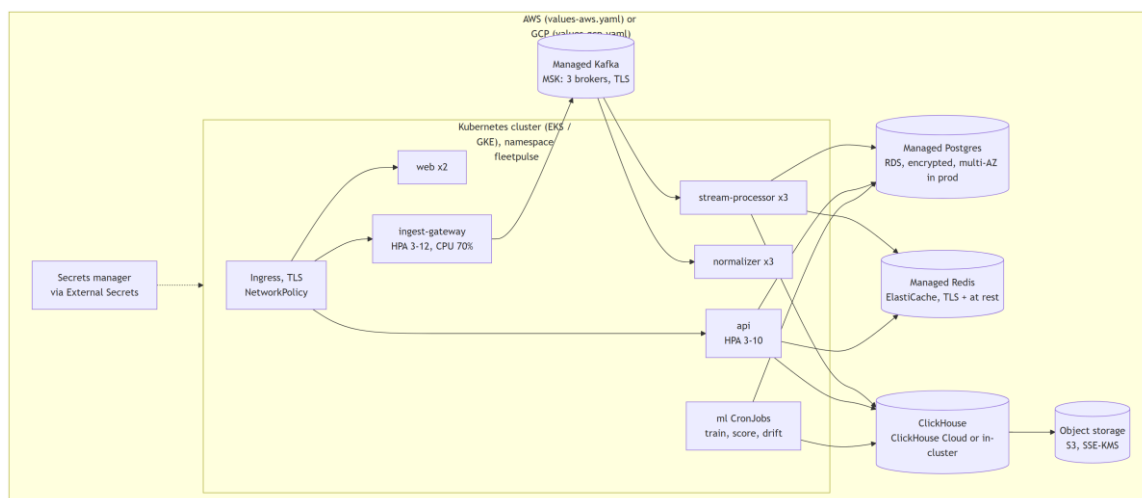


Figure 6. Production deployment: stateless services in Kubernetes, data in managed services.

- **Cluster:** one namespace; gateway HPA 3–12 and API HPA 3–10 at 70% CPU; normaliser and stream processor 3 replicas (scale with partitions, up to 24); replicas spread across zones; ingress with TLS and a NetworkPolicy.
- **Hardening:** non-root images, read-only root filesystem, all capabilities dropped, seccomp.
- **Secrets:** never in values; each is a key in one Kubernetes secret created by the platform (External Secrets, sealed secrets or the cloud's CSI driver).
- **Cloud-agnostic approach:** services only speak Kafka, Postgres, ClickHouse HTTP, Redis and the S3 API, configured by environment variables. `values-aws.yaml` and `values-gcp.yaml` differ; the code and images do not. Locally, `docker compose up -d --wait` runs the same images. Terraform (AWS) provisions EKS, RDS, MSK, ElastiCache, S3 and ECR with encryption. `helm lint`, `helm template` and `terraform validate` pass; **neither has been applied to a real cloud account.**

6. Low-Level Design

6.1 Layering & Separation of Concerns

Architecture style: event-driven microservices between services; **ports and adapters** inside them. The core logic (mapping engine, VIN and fault-code parsing, service-plan allocation, parts forecast, model

health) is pure code with no I/O, so it is unit-tested without brokers or databases, and adapters (Kafka, JDBC, HTTP, Redis) sit at the edges.

Layer	Responsibility (where in FleetPulse)	Must Not
Presentation / API	HTTP, SSE, validation, OIDC checks, RFC 9457 errors: <code>fleetpulse_api/routers/*</code> , <code>auth.py</code> , <code>errors.py</code> ; <code>IngestController.java</code>	Contain business rules. Shortcut taken: read-only list queries sit in routers (listed as debt)
Application / Service	Use cases and orchestration: <code>maintenance.py</code> (plan, forecast, model health, pure functions), <code>copilot/engine.py</code> , Kafka Streams processors	Depend on a specific database or broker
Domain	Entities, value objects, rules: <code>services/common</code> (CanonicalEvent, Vin, Dtc, MappingSpec, EventMapper), <code>AlertRule/RuleEngine</code>	Import framework or infrastructure code
Infrastructure	Adapters: <code>db.py</code> (tenant transactions), <code>clickhouse.py</code> , <code>ratelimit.py</code> (Redis), <code>RawPublisher</code> , <code>HttpClickHouseWriter</code> , <code>MappingRegistry</code>	Leak vendor types into the domain

Dependency direction: presentation → application → domain ← infrastructure (adapters implement ports such as `RawPublisher`, which the gateway's tests replace with a mock).

Folder structure (two levels)

```

fleetpulse/
├── services/
│   ├── common/           Java: canonical event, VIN/DTC parsing, mapping engine
│   ├── simulator/        Java: 100K vehicles, 4 formats, faults, backfill
│   ├── ingest-gateway/   Java: HTTPS intake, back-pressure, circuit breaker
│   ├── normalizer/       Java: Kafka Streams, dedup, DLQ, replay
│   ├── stream-processor/ Java: alert rules, Redis live state, ClickHouse sink
│   ├── api/              Python: FastAPI, copilot, maintenance planning
│   └── ml/                Python: features, training, scoring, drift
├── web/                  React dashboard (src/pages, src/components)
├── infra/                postgres/, clickhouse/, kafka/, keycloak/, onboarding/
├── deploy/               helm/fleetpulse/, terraform/aws/
├── tests/                chaos/, load/ (+ results/), bdd/
├── docs/                 adr/, evidence/, threat-model.md
├── scripts/demo/
├── docker-compose.yml
└── .github/workflows/ci.yml

```

6.2 Design Principles Applied

- **SOLID.** Single responsibility: each service has one job and Transform only converts units. Open/closed: a new maker is a new MappingSpec, and EventMapper is not modified. Interface segregation and dependency inversion: the gateway depends on the RawPublisher interface and the simulator on Sink (HTTP or Kafka implementations).
- **12-factor.** Config only from environment (`{...}` in `application.yml`, `API_*` via `pydantic-settings`); stateless processes (Kafka Streams state is rebuilt from changelogs); disposability (graceful close, proven by kill tests); logs to stdout; the same images in development and production.
- **Idempotency.** (VIN, seq) de-duplication window; alert `dedup_key` with `ON CONFLICT DO NOTHING`; ClickHouse insert tokens; idempotent Kafka producers.
- **Fail-fast.** Mapping specs are compiled and rejected at proposal time, not on live traffic; ClickHouse row policy errors if the tenant setting is missing instead of returning everything.
- **Least privilege.** `fleet_app` (subject to RLS) vs `fleet_service` (pipeline); read-only ClickHouse user for the API; the copilot can only propose; containers run as non-root.

- **DRY and KISS.** One mapping engine serves the gateway, normaliser, onboarding preview and the simulator's contract test. The service plan is a greedy allocation that a manager can follow, not a solver.

6.3 Design Patterns Used

Pattern	Problem It Solves in Our System	Location in Code
Adapter (declarative)	Maps each maker's payload to one canonical event	common/.../mapping/EventManager.java, PathExpr.java, mappings/*.json
Circuit Breaker + Retry with back-off	Fast 503s while Kafka is down; sends retry after Retry-After and never drop	ingest-gateway/CircuitBreaker.java; simulator/run/HttpSink.java
Idempotent consumer	Duplicates from retries are dropped exactly once	normalizer/SeqWindow.java, NormalizerTopology.java
Intent log (outbox-like)	Reproduce the same ClickHouse batch after a crash so rollups never double-count	stream-processor/.../sink/TelemetryBatcher.java (ADR 0001)
CQRS read models	Writes go to Kafka; reads from purpose-built views (Redis map, ClickHouse rollups)	stream-processor/.../live/LiveStateSink.java; 01_telemetry.sql
Observer	Every instance switches to a new mapping when it is activated	common/.../MappingRegistry.java (onActivated listeners)
Strategy	Unit and format conversions chosen per field	common/.../mapping/Transform.java
Token bucket	Per-user rate limit shared by all API replicas	fleetpulse_api/ratelimit.py (Redis Lua)
Proposal / approval (four-eyes)	The AI agent prepares, a human with the right role executes	copilot/tools.py, routers/copilot.py

6.4 Interfaces, Contracts & Runtime Flows

- **API contract:** OpenAPI generated by FastAPI at `/openapi.json` (interactive at `/docs`). Versioned under `/v1`. Keyset pagination: `?limit=` (max 500) and `?cursor=`, responses `{items, next_cursor}`. Errors are RFC 9457 problem details; another tenant's id returns the same 404 as a missing one. Rate limit 20 requests/s per user, burst 60, with `X-RateLimit-*` headers and `429 + Retry-After`.
- **Ingest contract:** POST `/v1/ingest/{oem}`, JSON array $\leq 1,000$ events, `X-API-Key`; 202 means durable in Kafka.

Topic	Key	Format	Partitions / retention
telemetry.raw	VIN (null until a maker is onboarded)	Maker's own JSON; headers: oem, request_id, received_at	24 / 7 days
telemetry.canonical	VIN	Canonical JSON with <code>schemaVersion = 1</code>	24 / 3 days
telemetry.dlq	original key	Original payload; headers <code>dlq_reason</code> , <code>dlq_detail</code>	6 / 14 days
oem.mappings	maker code	MappingSpec JSON, compacted (latest version wins)	1 / compacted
alerts	VIN	Alert JSON with deterministic dedup key	6 / 7 days

Schema evolution. Canonical fields are only added, as optional; consumers ignore unknown fields. A breaking change would raise `schemaVersion` and run both versions side by side. A maker's own format change is absorbed by a new mapping version, not by consumers. JSON was chosen over Avro to keep the time-boxed build simple; a schema registry with Avro is a future step.

Flow 1: the copilot proposes, a manager approves

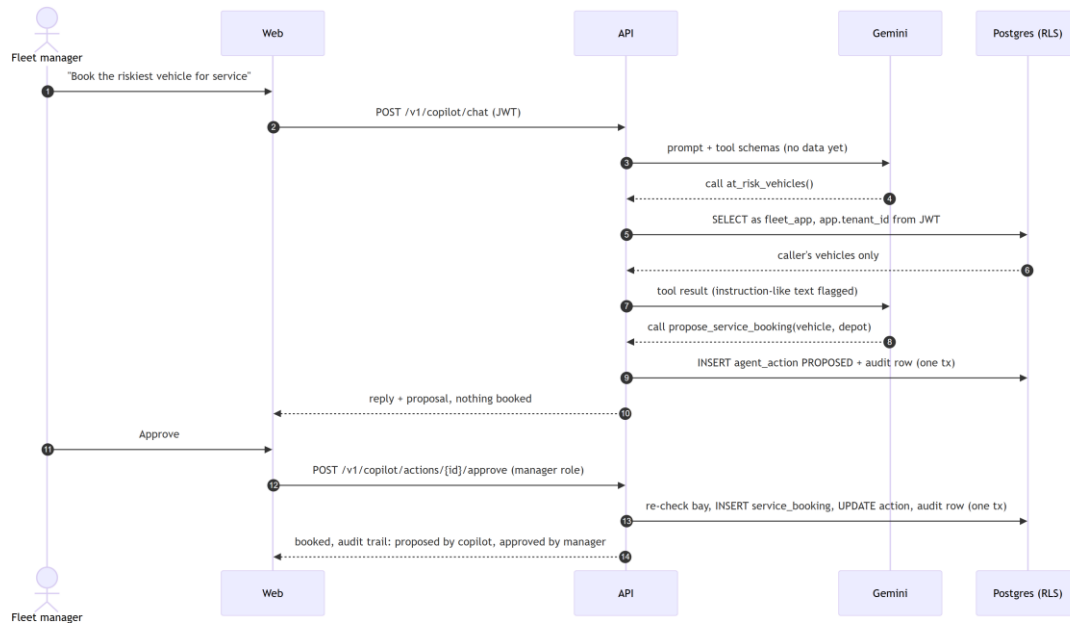


Figure 7. The model can only create a proposal; booking needs the manager role and commits with its audit row.

Flow 2 (failure path): a Kafka broker dies mid-stream

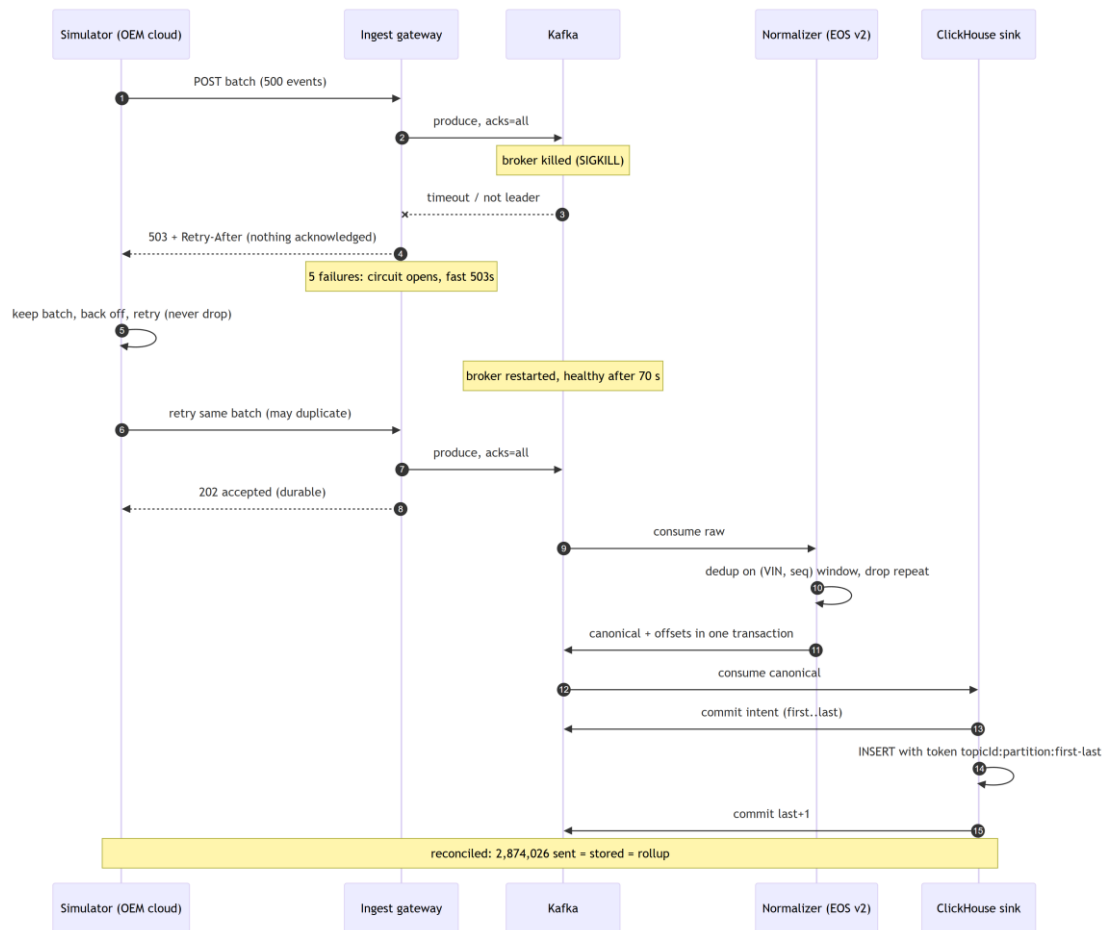


Figure 8. Nothing is acknowledged that is not in Kafka; retries are absorbed by de-duplication and insert tokens.

6.5 Algorithms & Data Structures

Problem	Algorithm / structure	Time	Space	Scale tested
Validate VINs	Transliteration table + positional weights, mod 11 check digit	$O(17)$	$O(1)$	Every event; 5,000 generated VINs in tests
Parse fault codes (text or raw bus bytes)	Regex for text; bit fields for 2-byte raw codes	$O(n)$ text, $O(1)$ raw	$O(k)$	Round trip over the 16-bit space in tests
Read any maker's payload	Paths compiled once to segments (field, index, filter, wildcard)	$O(\text{path length})$	$O(\text{path})$	400 events $\times$ 8 models contract test; every live event
De-duplicate (VIN, seq)	Per-vehicle ring buffer of the last 64 sequence numbers in a RocksDB store	$O(64)$ per event	~ 516 B/vehicle $\rightarrow$ ~ 52 MB for 100K	2.87M events reconciled after a broker kill
Rank at-risk vehicles	Composite B-tree index scan stopping at the page size	$O(\log n + k)$	index	98,000 scores: 2.3 ms
Paginate	Keyset cursor on the sort key	$O(\log n + k)$	$O(1)$	Page 900: 0.6 ms vs 140 ms
Plan inspections within bays	Greedy: urgent first, then by expected value; home depot then nearest within 350 km	$O(n \log n + n \cdot d)$	$O(n + d\text{-days})$	Acme: 491 inspections, 5 depots
Forecast parts	Sum of Bernoulli probabilities, 90% range from variance $\sum p(1-p)$	$O(n)$	$O(\text{depots} \times \text{parts})$	50K vehicles per tenant
Detect feed drift	Population stability index per maker vs the median of other makers	$O(n + \text{bins})$	$O(\text{bins})$	4 makers, 5-min windows
Rate limit	Token bucket in one Redis Lua script	$O(1)$	$O(\text{users})$	API load test

Problem	Algorithm / structure	Time	Space	Scale tested
Similar past failures	pgvector cosine distance on 16-number fingerprints	$O(n \cdot d)$ exact	$O(n \cdot d)$	Thousands of vectors
Geohash	Bit interleaving of lat/lon, base-32	$O(\text{precision})$	$O(1)$	Reference values in tests

Pseudocode: map and validate one payload (EventMapper.map)

```

for field in compiled_rules:           # compiled once per mapping version
    node = path.eval(payload)           # O(path length)
    value[field] = transform(node)       # unit / time / list conversion
require vin, seq, ts, lat, lon         # else MappingException(FIELD_MISSING)
check VIN check digit and maker prefix; ts within [now-7d, now+5min];
    coordinates in range and not (0,0); speed 0..300; percentages 0..100
dtc = unique(normalise(code) for code in value[dtc]) # text or raw hex
return CanonicalEvent(...)             # any failure → DLQ with a stable reason

```

Pseudocode: de-duplicate (SeqWindow.add)

```

window = store.get(vin) or empty ring of 64
if seq in window: drop as duplicate      # linear scan, cache-resident
window[next] = seq; next = (next + 1) mod 64
store.put(vin, window)                  # same Kafka transaction as the output

```

Pseudocode: service plan (maintenance.plan)

```

worth = [v for v in candidates if p(v) * saving(v) > inspection_cost]
sort worth by (not urgent(p >= 0.8), -expected_value, vehicle_id)
for v in worth:
    days = [tomorrow] if urgent(v) else plan_days
    for depot in [home(v)] + nearby(home(v), <= 350 km):
        for day in days: if free_bays(depot, day) > 0: assign; break
    if unassigned and urgent(v): report 'too risky to wait'

```

7. Non-Functional Requirements & Performance Benchmarks

NFR	Target (Case Study)	Achieved	How Measured
Ingest Throughput	100K+ events/sec	Not reached on one laptop. Kafka alone: 40,900 records/s (400 B, acks=all). End to end: 3,300/s sustained; 8,710/s stored during a 10K/s burst; up to ~11,000/s draining	tests/load/throughput.sh; Docker Desktop 8 GB, 12 CPUs; one broker
End-to-End Latency	< 2 s dashboard; < 5 s critical alert	Critical alert p50 0.44 s, p99 0.63 s. Dashboard: live state within ~1 s, page refresh every 5 s	58 alerts in steady state; reading timestamp → stored alert
API Latency	p95 < 200 ms; p99 < 500 ms	Not met on this laptop. List endpoints p50 75–87 ms, p95 0.36–1.0 s; fleet summary and maintenance overview p50 1.1–1.4 s; all requests p95 2.2 s, p99 4.5 s; 2 of 541 timed out (HTTP 500)	tests/load/api_latency.py, 2026-10-01 09:20 UTC: 3 users, 2 tenants, 120 s, while 100K vehicles stream and the pipeline saturates the laptop's CPU (results file in tests/load/results/)
Resilience	Recovers after broker / pod failure	Broker SIGKILL: serving again in 70 s; 2,874,026 sent = stored = rollup. Stream processor kill: sent = stored, no duplicates. Whole stack healthy 129 s after an unclean shutdown	tests/chaos/broker-kill.sh; ADR 0001; manual reconcile 6,348,000 across two restarts
Availability	99.9%, no single point of failure	Not demonstrated locally (one broker, one replica each). Production design: 3 brokers (RF 3, min ISR 2), ≥ 3 replicas per service across zones, managed multi-AZ stores	Helm values, Terraform; not run in a cloud

Load test setup. One laptop (Ryzen 5 5500U, 16 GB; Docker Desktop 8 GB, 12 CPUs), the full compose stack, 100,000 simulated vehicles. Steady: one report per vehicle every 30 s (3,300/s) for 5 minutes.

Burst: $\times 3$ to 10K/s for 2 minutes. Kafka alone: kafka-producer-perf-test, 2M $\times$ 400 B, acks=all, 24 partitions.

Burst run: t (s)	12	25	38	52	65	78	91	104	116	129
Generated / s	14,501	8,719	10,664	9,715	9,997	9,847	10,462	9,844	9,776	3,280
Stored / s	10,004	6,264	6,225	8,686	10,441	9,125	8,436	7,368	10,510	9,619
Backlog	53,957	85,872	143,578	157,982	152,211	161,590	187,938	220,123	211,316	128,911

Reading the results. At 10K events/s the laptop stores about 87% of the incoming rate, so the backlog grows to ~220K events and drains after the burst; no events were dropped. Scaling to the 100K events/s target needs more partition consumers per service on more cores (the services are stateless and partitioned by VIN), which we could not demonstrate on this hardware.

8. Security & Compliance

Method: STRIDE per trust boundary; full table of 14 threats in docs/threat-model.md. Top five:

Threat (STRIDE)	Control built	Residual risk
Another tenant's data read through a bug (Information disclosure)	Tenant enforced inside each store: Postgres RLS from a transaction-local setting, ClickHouse row policy per query, tenant-scoped Redis keys; API role has no BYPASSRLS; integration tests expect 404 across tenants	Pipeline role bypasses RLS by design; only batch jobs use it
Forged or replayed token (Spoofing)	Keycloak OIDC with PKCE; signature (JWKS), issuer, audience and expiry checked on every request	Token theft from a compromised browser
Prompt injection makes the copilot act (Elevation of privilege)	Propose-only tools; approval is a separate endpoint needing the manager role; tools run as the user under RLS; instruction-like text flagged; ≤ 3 proposals per turn	Pattern detection misses novel phrasings; the structural control is relied on
Lost or duplicated telemetry (Tampering / Repudiation)	acks=all, exactly-once streams, idempotent sinks; verified by kill tests	Not verified for multi-broker partitions or disk loss
'I never approved that' (Repudiation)	Hash-chained, append-only audit log written in the same transaction as each action; verification endpoint	Proves tampering, not who did it; anchor the head hash off-site next

- **Authentication and authorisation:** OAuth2/OIDC (Keycloak), RS256 JWT; roles viewer, manager and platform admin; tenant from the verified token only (ADR 0002).
- **Device identity and encryption:** OEM partners authenticate with an API key compared as a SHA-256 hash in constant time. **mTLS is not built** (data comes from OEM clouds, not devices; per-partner keys with rotation, or mTLS at the edge, are next). TLS terminates at the ingress in Kubernetes; locally services talk plain HTTP. At rest, Terraform enables RDS storage encryption, ElastiCache at-rest and in-transit encryption, MSK in-transit encryption and S3 SSE-KMS; the local stack is unencrypted. Secrets come from environment variables; .env is git-ignored; CI scans the whole history for secrets.
- **Privacy (GDPR, DPDP Act 2023):** driver personal data lives only in driver, never in telemetry; licence numbers are stored as salted hashes. Retention: Kafka raw 7 days, telemetry 90 days. **Location masking by role is not built, and the erasure endpoint is not built** (the schema has nullable personal fields and an erasure_request table with a receipt). All data in this submission is synthetic.

- **AI safety:** propose-only tools, human approval by role, tenant-scoped tools, injection flagging, proposal cap, audit trail per proposal and decision. When a Gemini key is configured, tool results leave for Google; production would need a data-processing agreement or a self-hosted model.

9. Test Strategy

Test Type	Tools	No. of Tests	Coverage / Result	In CI?
Unit	JUnit 5, AssertJ, Mockito, Kafka Streams TopologyTestDriver; pytest; Vitest	Java 117; API unit + integration 72 (+1 skipped); ML 21; web 6	All pass (2026-10-01). Line coverage: Java 49.2% overall (common 90.2%, gateway 63.5%, stream processor 50.2%, normaliser 32.2%, simulator 30.4%); API 77%; ML 35%	Yes (API integration tests skip in CI)
Integration & Contract	pytest against the running stack with real Keycloak tokens; simulator ↔ normaliser contract test	API integration suite (tenant isolation, booking conflicts, copilot approval, audit); contract: 8 models × 400 events	Pass. Pact not used; Testcontainers not used (integration runs against compose)	Contract yes; API integration locally
Acceptance (BDD)	–	0	Not done	–
Performance / Load / Soak	kafka-producer-perf-test, tests/load/throughput.sh, tests/load/api_latency.py	3 scripts	Section 7. Steady state 5 min; no long soak	No (manual)
Security (SAST, DAST, Dependency, Image)	gitleaks (history), Trivy fs (vulnerabilities, IaC misconfiguration, secrets), Trivy image scan per image, ruff flake8-bandit rules (S)	Every push	Pipeline fails on critical/high fixable findings; Semgrep and OWASP ZAP not run	Yes
Compliance & Chaos	tests/chaos/broker-kill.sh; stream-processor kill; audit chain verification endpoint	Broker kill, processor kill, unclean stack restart	PASS: 2,874,026 sent = stored = rollout; erasure flow not tested (not built)	No (manual)

Edge cases covered: duplicate and late events (dedup window, event-time ordering); invalid VINs, null-island GPS, out-of-range values and malformed JSON (DLQ with reasons); an unknown maker (parked, then replayed); broker outage (503, circuit breaker, retry); a crash between each step of the ClickHouse sink; another tenant's ids (404); booking conflicts (all or nothing); prompt-injection text in tool results; broken sensors; label-boundary and leakage tests for the model.

Sample input / output: a Borealis payload

```
{"vehicle":{"id":...},"spd_mph":40,"ect_f":212,"faults":"P0217,P0128","ev_code":"HA"} becomes speedKmh 64.4, coolantC 100.0, dtc [P0217,P0128], evt HARSH_ACCEL (EventMapperTest).
```

Report links: JaCoCo services/*/target/site/jacoco/, Surefire services/*/target/surefire-reports/, load and chaos results in tests/*/results/, security scans in the CI run <https://github.com/vennela9927/fleetPulseproject/actions>.

10. Observability

What exists. Every Java service exposes Prometheus metrics at /actuator/prometheus (gateway batch latency p50/p95/p99, pending records, rejections by status; normaliser events, DLQ and

duplicates by maker and reason; Kafka Streams client metrics; simulator ledger and tick lag). The API exposes `/metrics`. Logs are structured, to stdout. Consumer lag is visible in Kafka UI (optional compose profile), and the Chaos page reconciles sent and stored per maker live. **Not built:** Grafana dashboards, OpenTelemetry tracing and centralised logs (ELK or Splunk).

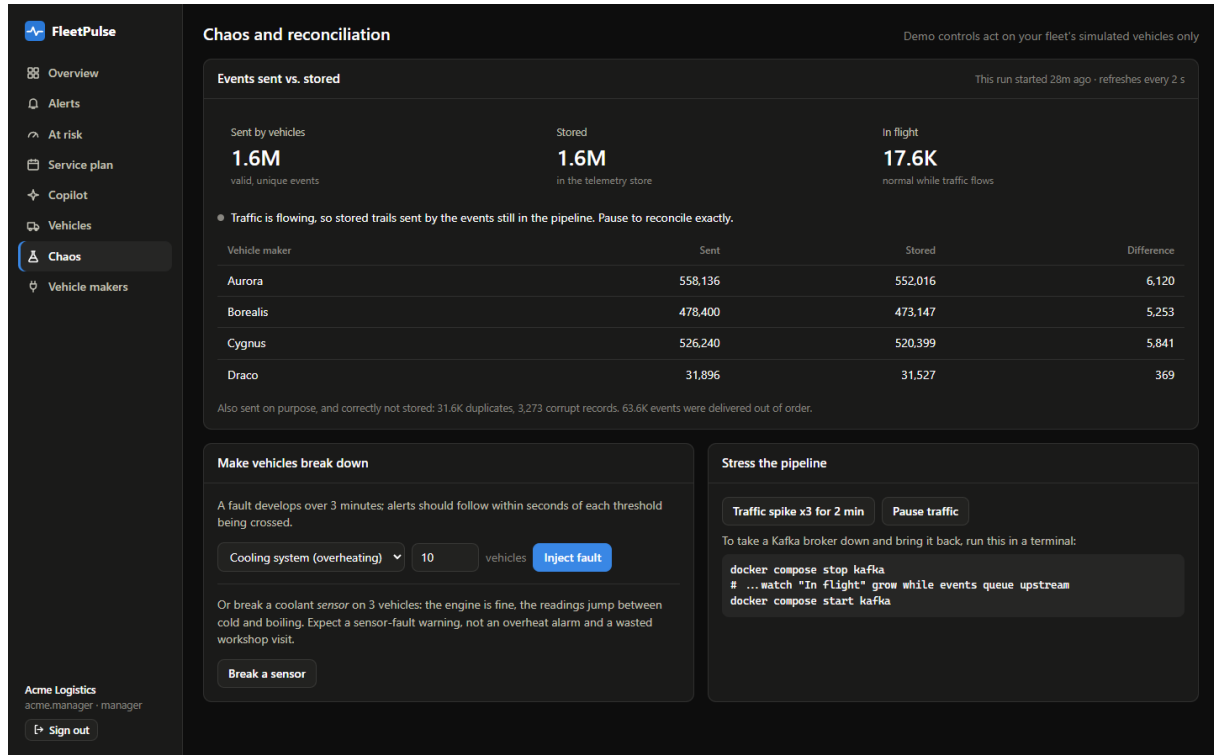


Figure 12. Chaos page: the reconciliation view used as the pipeline's health dashboard (in flight = sent – stored).

Troubleshooting a latency spike (walk-through).

- Is it intake? `gateway_batch_latency p99` and `gateway_pending_records` near 50,000, plus 429/503 counts by maker in `gateway_rejected_total`, mean Kafka is slow to acknowledge.
- Is it processing? Consumer lag per group (normalizer, alert-detector, live-state, clickhouse-sink) shows which stage falls behind; Kafka Streams metrics show commit latency and rebalances.
- Is it storage? The sink's batch insert times and ClickHouse `system.parts / system.merges` show merge pressure (the cause of one real incident: thousands of small parts).
- Logs by request id (the gateway's `X-Request-Id` travels as a Kafka header) tie a slow batch to its records. Without tracing this step is manual, which is why OpenTelemetry is the next observability step.

11. AI / ML Component (if used)

Purpose. Decide which vehicles to inspect, when and where. Rules alone were not enough: at the same number of vehicles flagged per day, the mechanic's rule caught 54% of breakdowns and lost money; the model caught 80%.

Data and features. 21 days of daily rollups for 98,000 vehicles with 3,669 breakdowns, generated by our simulator. For each vehicle and day, features use only the 7 days up to that day: levels, 3-day

trends, days with each fault code. As-of days are limited to fully observed 7-day label windows. Tests guard against leakage and shifting label boundaries.

Model and agent design. LightGBM, calibrated with isotonic regression on held-out vehicles (capped at 99%); per-feature contributions are each vehicle's reasons. The copilot uses Gemini function calling (models tried in order, rule-based fallback) with 10 tools: 8 read tools scoped to the user's tenant (including similar past failures from pgvector) and 2 propose-only tools.

Evaluation (20% unseen vehicles, later days)	Mechanic's rule	Model, same number of checks	Model, cost rule
Flagged vehicles that broke down	8.0%	11.9%	77.8%
Breakdowns caught	53.9%	80.1%	78.0%
Median warning	2.1 days	2.7 days	2.7 days
Net saving per week per 100K vehicles	-\$466K	-\$303K	+\$712K

PR-AUC 0.748 against a 0.012 base rate; ROC-AUC 0.89; likely part correct 83% of the time; predictions of 80–100% came true 93.6% of the time. Leaving each maker out of training in turn lowered PR-AUC by at most 0.011 (Aurora 0.77→0.76, Borealis 0.70→0.69, Cygnus 0.77→0.77).

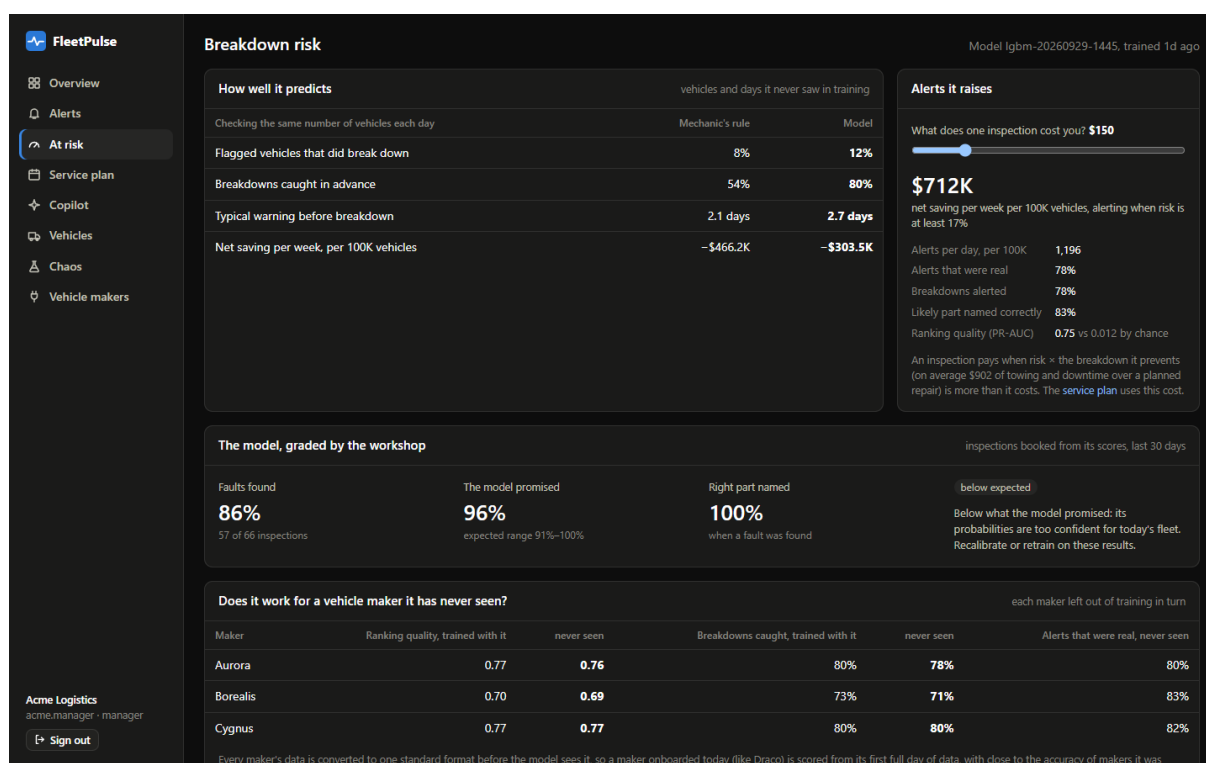


Figure 9. Breakdown risk page: the back-test, the cost slider, workshop grading and the never-seen-maker check, as shown to the manager.

Guardrails and cost. Calibration plus workshop grading detect overconfidence (it flagged the 99% cap). The copilot cannot book anything; every proposal and decision is audited; without a key or when Gemini is unavailable the rule-based assistant answers with the same approval flow. Training takes about a minute; scoring runs daily. Copilot replies take about 1 s with gemini-3.5-flash-lite on the free tier (\$0 in this project; token usage was not metered).

12. Architecture Decisions, Risks & Future Enhancements

ADR	Context → options → decision → consequences
0001 Exactly-once into ClickHouse	Rollups double-count a re-inserted batch → options: FINAL on reads, Kafka Connect with Keeper, ClickHouse transactions → record each batch's boundary in Kafka before inserting, with a dedup token → two offset commits per batch; 30 minutes of downtime covered by the dedup window.
0002 Tenant isolation in every store	One missing WHERE would leak a fleet → options: app filtering, database per tenant → RLS, ClickHouse row policies, tenant keys in Redis → a bug becomes missing data, not a breach; one set_config per request.
0003 Failure prediction model	Rare events (1.2%), need 'how sure' → options: survival, sequence models, precision threshold → calibrated LightGBM with a cost rule → explainable, maker-independent; 2.7-day median warning.
0004 Trusting data and model after launch	Sensors fail, feeds drift, the world moves → sensor plausibility, relative per-maker PSI, workshop grading → drift silent for an hour after a gap; coolant only for now.
0005 Polyglot persistence (CAP)	Four access patterns → Postgres (CP), ClickHouse and Redis (AP), pgvector → four stores to operate, each available as a managed service.

Risks and technical debt:

- All data is simulated; results show the method works on realistic structure, not what a given fleet would save.
- Local runs use one Kafka broker; the laptop sustains 3,300 events/s steady and ~8,700/s in bursts, below the 100K events/s target. After a laptop sleep, the normaliser's backlog drained at only ~1,400 events/s on 2026-10-01: throughput under backlog needs profiling.
- API latency misses the target on the laptop (p95 2.2 s overall, two timeouts): the fleet summary and maintenance overview recompute per request while the pipeline saturates the CPU. Next: precompute both per tenant every few seconds into Redis, and run the API on its own nodes.
- Line coverage below 80% outside the core library; no BDD suite, Semgrep, ZAP or Pact; no Grafana or tracing.
- Location masking, the erasure endpoint, per-partner keys and mTLS are not built; read-only SQL sits in API routers.

Future enhancements (prototype → pilot):

- Run on the Helm chart with 3 brokers and managed stores; repeat the load test at 100K events/s with more partition consumers, and the chaos tests with multi-broker failures.
- Pilot with one real fleet's maker feeds and workshop records: retrain on real outcomes, use the workshop loop to decide recalibration.
- Close the compliance gaps: erasure endpoint, location masking by role, per-partner keys with rotation, Avro schemas in a registry, and dashboards with tracing.

13. Demo Video (5 Minutes Maximum)

Time	Segment	What We Show
0:00 – 0:30	Problem	The fleet manager, maker-specific data and roadside breakdowns: about 1,200 a week per 100K vehicles in our simulated fleet, each about \$900 more than a planned repair

Time	Segment	What We Show
0:30 – 1:00	Solution	One-line pitch; on unseen data the model catches 80% of breakdowns against 54% for a mechanic's rule; +\$712K a week per 100K vehicles at a \$150 inspection
1:00 – 3:00	Live demo	Fleet health; the live alert feed (the real-time event); the first vehicle in line with its risk, part and reasons; the service plan within depot bays; the copilot proposes a booking, the manager approves, audit trail; vehicle makers and feed health
3:00 – 4:15	Under the hood	Architecture; scale on one laptop (3,300 events/s, critical alerts p99 0.63 s); the Chaos testing page (sent vs stored, bad data rejected) and the broker-kill result (2,874,026 sent = stored)
4:15 – 5:00	Impact & next steps	Measured results; next steps (real fleet data, a three-broker cloud run); the team

Video link: <https://drive.google.com/file/d/1xHAFBI4Agx50tX9YDKDF9llaxLwQ8bW9/view?usp=sharing>

14. Repository Checklist

Item	Status
README: problem, architecture, quick start, environment variables, tests, known issues	Done (README.md)
One-command run with the simulator: <code>docker compose up -d --wait</code>	Done; all services healthy in 129 s after an unclean shutdown
Structure: service folders, /docs, /infra, /tests	Done
CI on every push: build, lint, all unit suites, secret/dependency/laC scans, image scans, whole-stack smoke test	Done (.github/workflows/ci.yml); https://github.com/vennela9927/fleetPulse project/actions
Hygiene: no secrets committed, .env.example provided	Done; gitleaks scans history in CI; GEMINI_API_KEY empty in .env.example
Commits from all members	All commits so far are by one author (Vennela Reddy Vootkur)
Final tag v1.0-submission	Done: v1.0-submission

15. Conclusion

Key learnings. Exactly-once is a property of the whole path, not of Kafka: the ClickHouse rollups double-counted a retried batch until batch boundaries were recorded before inserting. On real hardware, the bottlenecks were not where we expected (healthchecks timing out under load, a single-threaded sink, memory limits).

Strengths. Depth on one problem: from messy multi-maker data to a booked, audited workshop plan; every claim measured and scoped; tenant isolation enforced by the databases; a new maker without code.

Challenges solved. MinIO images disappeared (replaced by SeaweedFS behind the S3 API); ClickHouse ran out of memory merging thousands of small backfill parts (day-ordered inserts); Kafka wrote outside its volume; a normaliser was killed for memory; rebalance storms from rebuilding uncompact changelogs; drift false alarms after restarts (relative rule, shorter baseline, continuity check); the model's 99% cap was overconfident (caught by workshop grading).

16. Declarations

Open-source components

Component	Licence	Use
Apache Kafka 3.8, Kafka Streams	Apache-2.0	Event log, stream processing
PostgreSQL 16 / pgvector	PostgreSQL License	Relational core, vectors
ClickHouse 24.8	Apache-2.0	Telemetry and rollups
Redis 7.4	RSALv2 / SSPLv1 (dual)	Live state; licence to be reviewed for commercial use (Valkey is a BSD alternative)
Keycloak 25	Apache-2.0	Identity provider
SeaweedFS	Apache-2.0	S3-compatible storage (local)
Spring Boot 3.3, Jackson, Micrometer	Apache-2.0	Java services
FastAPI, Pydantic, Uvicorn, httpx	MIT / BSD-3	API
psycopg 3	LGPL-3.0	Postgres driver
LightGBM / scikit-learn, pandas, NumPy	MIT / BSD-3	Model, calibration, data
React, Vite, TanStack Query, Recharts	MIT	Dashboard
MapLibre GL JS	BSD-3-Clause	Live map
kafka-ui (kafbat), Adminer, RedisInsight	Apache-2.0 / Apache-2.0 / SSPL	Optional development tools

AI tools used

- **Claude Code (Anthropic, Claude Opus model):** design discussion, code generation for all services, tests, infrastructure files, documentation and this document's drafting, with every result run, measured and reviewed by the team.
- **Gemini API (Google):** at runtime only, as the copilot's language model; optional.

Data

All data is synthetic: four fictional makers, generated VINs, driver names combined from name lists, example.test email addresses and simulated telemetry. No real personal or vehicle-owner data is used.

17. Appendix (if any)

Evidence in the repository

Evidence	Path
Broker-kill reconciliation (PASS)	tests/chaos/results/broker-kill-20260930T104323Z.txt
Throughput test (Kafka alone and end to end)	tests/load/results/throughput-20260930T104647Z.txt
API latency	tests/load/results/api-latency-*.txt (script tests/load/api_latency.py)
Query plans before and after	docs/evidence/query-plans.md
Architecture decisions	docs/adr/0001 – 0005
Threat model (STRIDE)	docs/threat-model.md
Diagram sources (Mermaid)	docs/diagrams/*.mmd

More screens

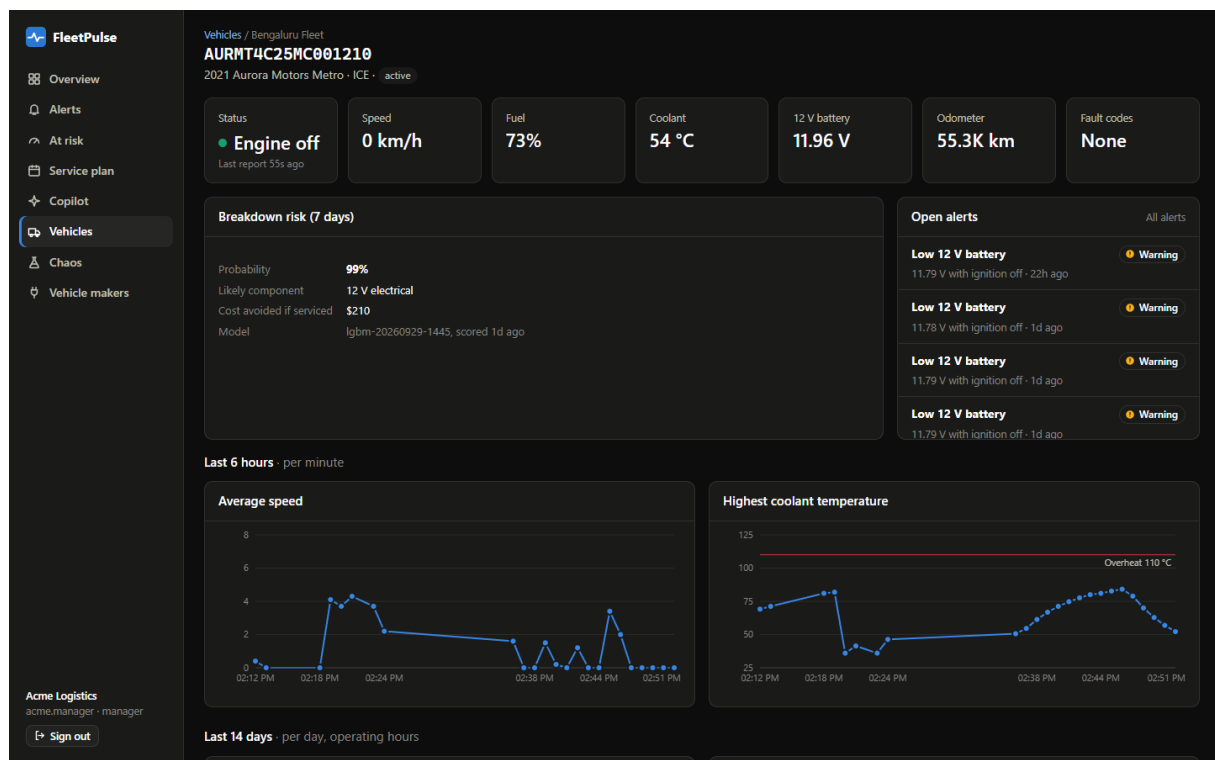


Figure 10. Vehicle page: live state, history, risk and its reasons.

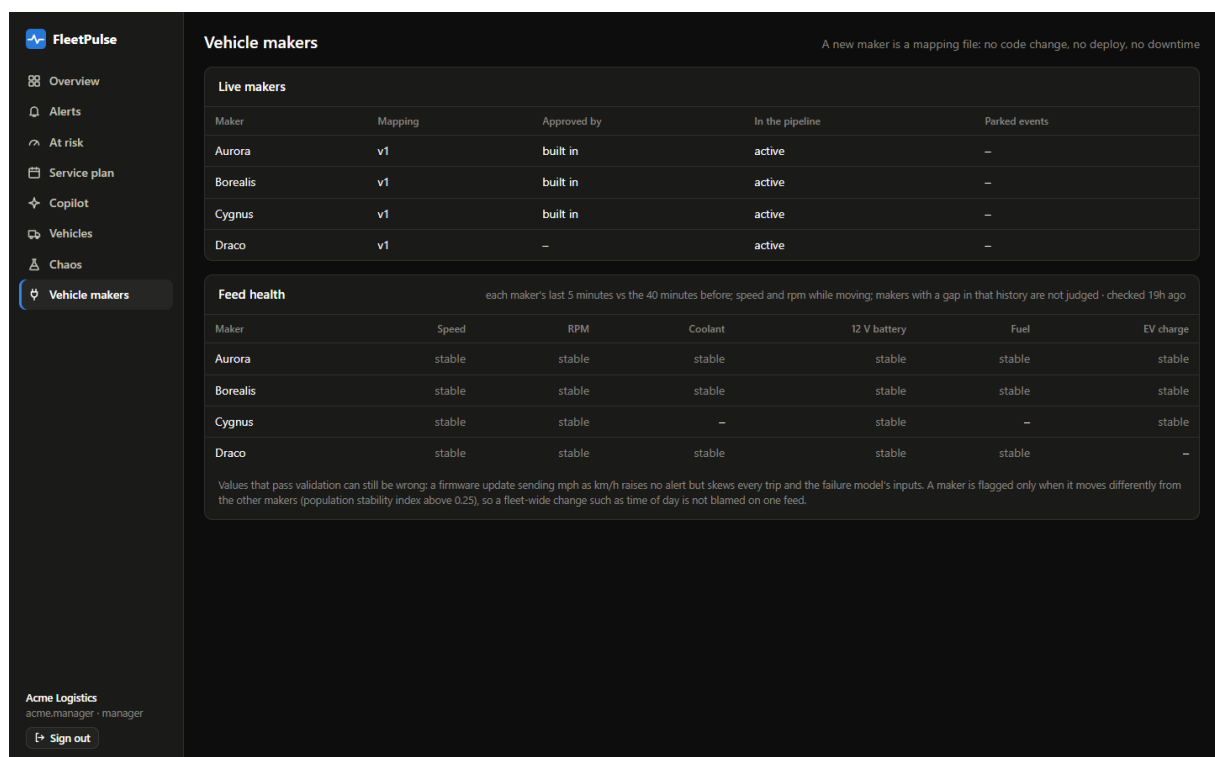


Figure 11. Vehicle makers: mapping versions and per-maker feed health.

References

- Connected Vehicle Intelligence Hackathon, Problem Statement & Case Study (Talenciaglobal), sections 2–14.

- ISO 3779 (VIN structure) and SAE J2012 (diagnostic trouble codes), as implemented in services/common.
- RFC 9457, Problem Details for HTTP APIs.